

programa de pós-graduação em
ENGENHARIA DE DEFESA

Álgebra Linear Computacional

Parte VII

Maj Hebert **AZEVEDO** Sá, Ph.D.

Computadores armazenam números em formatos que são compostos por números binários (*bits*).

No caso de números inteiros, o limite é dado pelo total de bits que se pode alocar para representá-los, de acordo com a arquitetura do processador.

Números do tipo “ponto-flutuante” tem precisão limitada. Portanto, algoritmos numéricos carregam erros inerentes de aproximação ou truncamento.

Representação de números inteiros

Com p bits podemos representar números sem sinal ou com sinal.

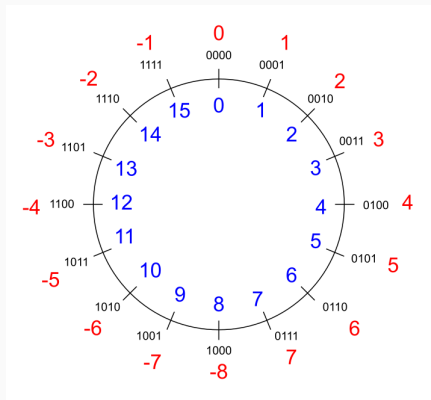
Para encontrar a representação de números positivos, basta fazer uma conversão de decimal para binário (com sucessivas divisões por 2).

Para encontrar a representação de números negativos, tiramos o **complemento-2** da representação de seu equivalente positivo. Para tirar o complemento dois, basta inverter todos os bits e somar 1 ao resultado da inversão.

Para p bits, podemos representar de -2^{p-1} até $(2^{p-1} - 1)$ ou de 0 até $(2^p - 1)$.

Representação de números inteiros

Exemplo: Para $p = 4$:



vermelho: números com sinal; azul: números sem sinal

Representação de números do tipo “ponto-flutuante”

As representações são, geralmente, divididas em:

- sinal
- base (b) - em computadores, $b = 2$.
- expoente (n)
- mantissa (inteiro composto por dígitos $d_1d_2...d_p$)

e são dadas por

$$\pm(0.d_1d_2...d_p) \times b^n$$

Representações mais comuns hoje em dia: 32 bits (*single precision*, sendo 1 para o sinal, 8 para o expoente e 23 para a mantissa) ou 64 bits (*double precision*, sendo 1 para o sinal, 11 para o expoente e 52 para a mantissa).

Representação de números do tipo “ponto-flutuante”

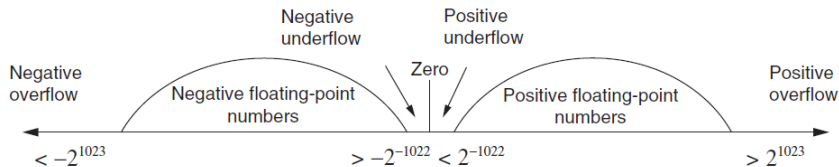


FIGURE 8.2 Map of IEEE double-precision floating-point.

Representação de números do tipo “ponto-flutuante”

Naturalmente há limites de precisão na representação de números reais com pontos-flutuantes. Há lacunas entre os pontos-flutuantes, por exemplo: em precisão dupla (64 bits) o ponto-flutuante mais próximo e maior que 1 é $1 + 2^{-52}$. A *precisão da máquina* (*eps*) é 2^{-52} .

Em Python:

```
1  # Import numpy
2  import numpy as np
3
4  # Print the machine epsilon
5  print("Machine epsilon:\n",np.finfo(float).eps)
```

As operações computacionais são aproximações das operações matemáticas tradicionais.

Dependendo do resultado, é possível ocorrer *overflow* (quando é produzido um número maior do que se pode representar) ou *underflow* (quando é produzido um número menor do que se pode representar).

Como minimizar erros de computação?

Algumas dicas:

Evitar adições entre números “pequenos” e números “grandes”

Evitar subtrações entre números “muito próximos”

Exemplos: Ford, Exemplos 8.8 e 8.9

Há métodos para medirmos o nível de complexidade e esforço computacional de algoritmos de forma normalizada. O método mais comum é contabilizar o número de operações de pontos-flutuantes (floating-point operations - *flops*).

A complexidade de um algoritmo é dada pela ordem de grandeza do número de flops dele. Usamos a notação “O-grande” para representá-la. Para encontrar a complexidade, é necessário calcular o número de flops em função da ordem dos elementos de entrada (matrizes, vetores, etc.) e encontrar os termos dominantes.

Exemplo: um algoritmo que requeira $\frac{4}{3}n^3 + 9n^2 + 8n + 6$ flops é um algoritmo de complexidade $O(n^3)$.

Exemplo: o cálculo do produto interno de 2 vetores de ordem n requer n multiplicações e $(n - 1)$ adições, i.e., $(2n - 1)$ flops. Esse algoritmo é $O(n)$.

Exemplo: o cálculo do produto entre uma matriz $m \times p$ e uma matriz $p \times n$ é $O(mnp)$. Se ambas matrizes são quadradas de ordem n , o algoritmo será $O(n^3)$.

Exercício: Qual é a complexidade deste algoritmo de substituição regressiva?

Algorithm 9.4 Solving an Upper Triangular System

```
function BACKSOLVE(U,b)
% Find the solution to  $Ux = b$ , where U is an  $n \times n$  upper-triangular matrix.
 $x_n = b_n / u_{nn}$ 
for i = n-1:-1:1 do
    sum = 0.0
    for j = i+1:n do
         $sum = sum + u_{ij}x_j$ 
    end for
     $x(i) = (b(i) - sum) / u_{ii}$ 
end for
return x
end function
```

R: $O(n^2)$